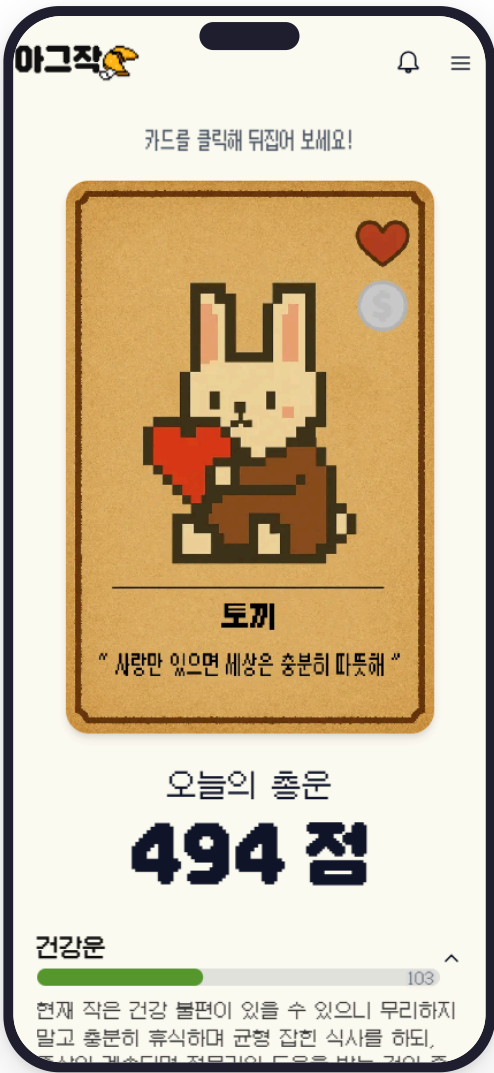
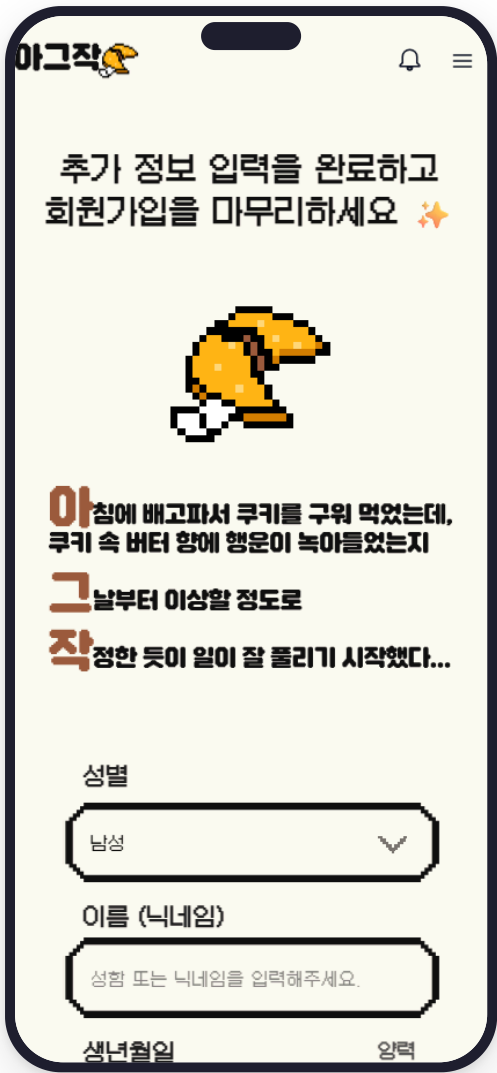




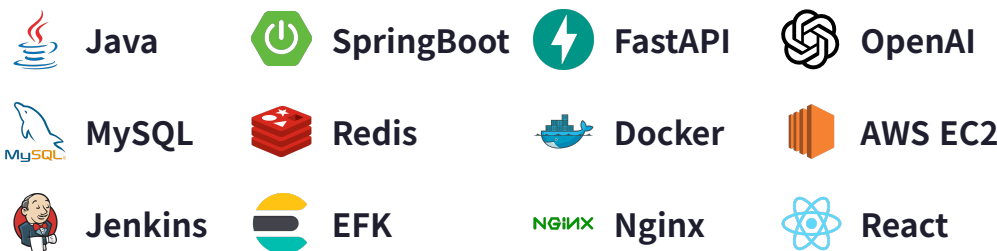
서비스

운세 강화게임이 결합된 데일리 운세 플랫폼 자율 PJT 우수상 2등

AI 기반 운세 계산·강화·공유 + 포춘쿠키 수집·랭킹 시스템을 결합한 운세 강화 게임 & 소셜 웹 서비스

[GitHub Repository](#)

기술 스택



역할 — Backend

인증/보안

OAuth2 + JWT, 토큰 회전(RTR) 및 CORS/SSL 최적화

랭킹 설계

Redis ZSET(실시간) + MySQL 스냅샷(영속성) 랭킹 이원화 구축

게임 로직

먹등성 키를 활용한 일일 제한(Cap) 및 동시성 제어

API 설계

공유 토큰 기반 딥링크 API 및 쿠키/도감 도메인 구현

협업 표준화

Swagger 기반 API 명세 및 DTO/에러 스펙 표준화

작업 기간

2025. 10. 13 - 2025. 11. 20 (6주)

SSAFY 13기 자율 프로젝트

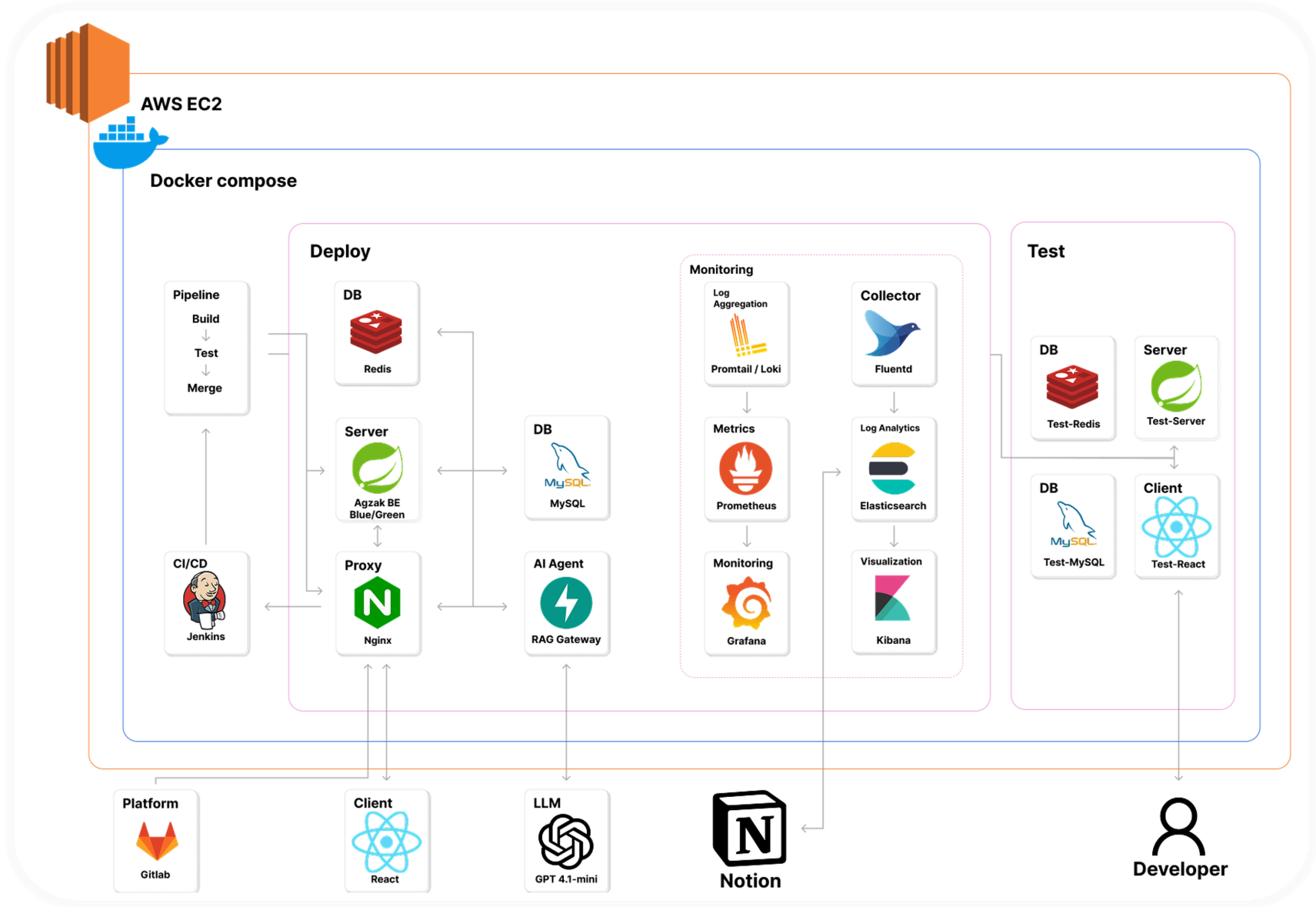
구성원

총 6명

FE 3명 · BE 2명 (본인포함) · Infra 1명

프로젝트 회고

- 실 트래픽 운영을 처음 받아본 경험 — 14일간 1,100명+ / 11만 이벤트
 - 수치로 안전했음을 증명할 수 있는 구조의 가치를 체감
- 환경이 만드는 장애를 처음 진단 — 스케줄러 데드락
 - 다중 인스턴스 + 자정 cron = 데드락
 - ShedLock + 매시간 분할로 발생률 0건 / 다음엔 예방적으로 잡기
- 같은 단어 안의 다른 요구를 분리 — 랭킹 이원화
 - Redis ZSET(실시간) + MySQL 스냅샷(영속)으로 Slow Query 0건
- 먹등성 = 비즈니스 정합성의 도구
 - 게임 일일 Cap에 먹등키 — 더블클릭·재시도에도 정확한 cap
- 방어를 코드로 녹였지만, 효과 증명은 숙제
 - JWT RTR + Redis 블랙리스트로 토큰 탈취 시나리오 방어
 - 다음엔 모의 침해 회귀 테스트로 효과 증명
- 모니터링은 질문하는 사람이 살아있게 한다
 - 대시보드를 매일 판독하며 "어떤 수치가 위험한가"를 묻는 습관 학습



① Deploy 환경

AWS EC2 · Blue/Green

- ▶ 헬스 엔드포인트(/actuator/health)로 트래픽 전환 트리거 제공
- ▶ ShedLock 으로 두 슬롯 동시 기동 안전성 보장 (스케줄러 중복 실행 차단)
- ▶ 운영/테스트 환경 동일 이미지 — 환경변수만으로 기동 분기
- ▶ Spring Boot 두 슬롯 + Nginx upstream swap 으로 무중단 배포

배포 다운타임 0초 · 동시 두 슬롯 안전

② CI / CD

Jenkins · 4 파이프라인

- ▶ Gradle 빌드 → Docker 이미지 → 레지스트리 푸시 자동화 흐름 호환
- ▶ health=UP 응답 보장으로 무중단 슬롯 컷오버
- ▶ 운영 동일 이미지로 테스트 환경(격리 DB/Redis) 회귀 검증 통과
- ▶ 4 파이프라인(운영·테스트 × BE·FE) + Mattermost 자동 알림

Docker 캐시 적중 시 빌드 1m 20s → 14s

③ 모니터링 · 로그

Prom · Graf · Loki · EFK

- ▶ Spring Actuator 메트릭 + trace_id 표준 로그 포맷 노출
- ▶ Mattermost 알람(5xx · Broken Pipe · Deadlock 40001) 추적·원인 규명
- ▶ 메트릭 — Prometheus + Grafana + Exporter (서버·MySQL·Nginx)
- ▶ 로그 — Loki/Promtail · EFK 병행 + Notion DB 일별 자동 리포트

14일 운영 Slow Query 0건 · ShedLock 적용 후 자정 데드락 재발 0건

④ AI 연동 · Test

FastAPI 게이트웨이 호출

- ▶ 운세 계산(결정론, Swissep) / 해설(AI 변동) 책임 분리 설계
- ▶ 항목별 AI 호출 병렬화 + 응답 개심으로 Broken Pipe 회복
- ▶ 외부 호출은 AI 게이트웨이 1곳으로 통제 — 내부 네트워크 호출만
- ▶ AI 게이트웨이 별도 컨테이너 (FastAPI · OpenAI · RAG · Swissep)

AI 호출 분리·병렬·캐싱 → Broken Pipe 회복

랭킹 저장소 이원화

Project - 1

PROBLEM → SOLUTION

"오늘 랭킹은 즉시 반영, 과거 랭킹은 그 시점 그대로 보존" — 두 요구는 **같은 자료구조로 풀 수 없다**. 갱신 빈도·정렬 비용·보존 요구가 다르므로 저장소 자체를 분리한다.

Before vs After

BEFORE

단일 MySQL + ORDER BY

- ✗ 매 요청 풀스캔 — 데이터 변경 없어도 매번 전체 read
- ✗ 세 가지 정렬 모두 메모리 — total / original / gain
- ✗ "내 등수" 추가 카운트 쿼리 — 결과를 다시 정렬
- ✗ 과거 일자 보존 X — Fortune 갱신 시 어제 순위 흔들림

AFTER

Redis ZSET + MySQL 스냅샷

- ✓ 점수 갱신 = ZADD $O(\log N)$ — DB 정렬보다 압도적
- ✓ TOP N = ZREVRANGE 단건 — 정렬 비용 0
- ✓ "내 등수" = ZREVRANK 한 번 — 별도 쿼리 X
- ✓ 과거 일자 = MySQL 스냅샷 영속 — 갱신에도 흔들리지 않음

동기화 전략 — ShedLock 2 스케줄 + Fallback



domain/ranking/service/RankingService.java

매시간 59:55 · ShedLock

```
@Scheduled(cron = "55 59 * * *", zone = "Asia/Seoul")
@SchedulerLock(name = "rankingSnapshot",
    lockAtMostFor = "PT4M", lockAtLeastFor = "PT20S")
public void todayRebuild() { rankingService.rebuildTodaySnapshot(); }

@Transactional
public void rebuildTodaySnapshot() {
    LocalDate today = redisStore.today();
    var all = redisStore.readTop(today, redisStore.zTotal(today),
        Integer.MAX_VALUE);

    // ★ 탈퇴 회원 prune - Redis 고아 데이터 정리
    Set<Long> ids = all.stream().map(UserRow::memberId).collect(toSet());
    Set<Long> alive = mysqlStore.existingMemberIds(ids);
    for (Long mid : ids)
        if (!alive.contains(mid)) redisStore.removeTodayUser(mid);

    // 살아있는 회원만 rank_no 1번부터 영속 저장
    int rank = 1;
    List<Ranking> rows = new ArrayList<>(alive.size());
    for (var r : all) {
        if (!alive.contains(r.memberId())) continue;
        rows.add(Ranking.builder()
            .member(mysqlStore.refMember(r.memberId())) // em.getReference (N+1 회피)
            .date(today).rankNo(rank++)
            .totalScore(r.total()).originalTotalScore(r.original())
            .build());
    }
    mysqlStore.replaceTodaySnapshot(today, rows); // deleteByDate + saveAll
}

// ★ 위기 fallback - Redis 죽어도 MySQL 스냅샷으로 자동 대체
public RankingListDto getTodayTopN(int limit) {
    try { /* Redis 경로 */ }
    catch (Exception e) { return getSnapshotTopN(d, limit); }
}
```

0건

Slow Query

14일 운영 · MySQL QPS 한 자리수 유지

$O(\log N)$

랭킹 갱신·조회

ZADD · ZREVRANGE · ZREVRANK

0초

캐시 TTL 대기

강화 즉시 랭킹 반영

3축

단일 장애점 회피

Fallback · Reseed · Prune

1.1천 명

활성 사용자

14일 (2025.11.11~24) · 신규 97%

11만

이벤트 수

강화 시도 / 운세 조회 / 쿠키 굶기

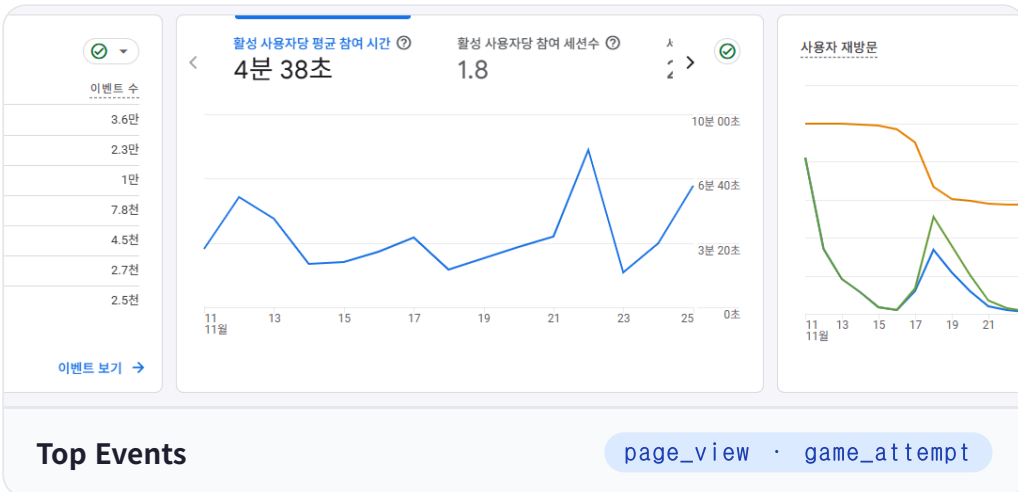
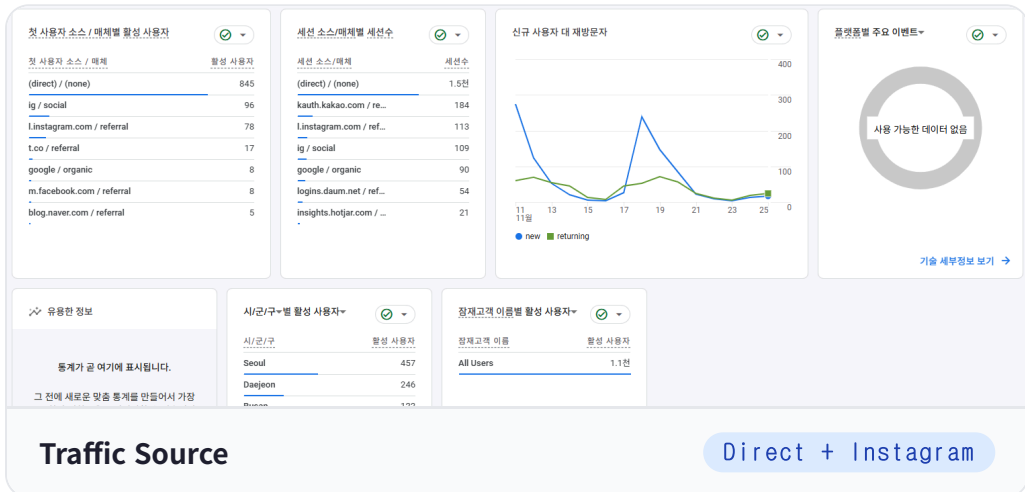
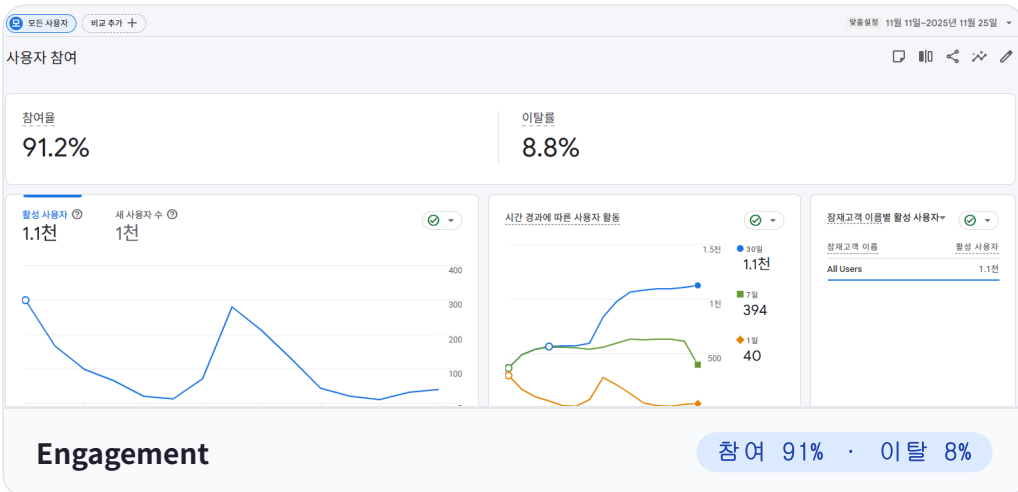
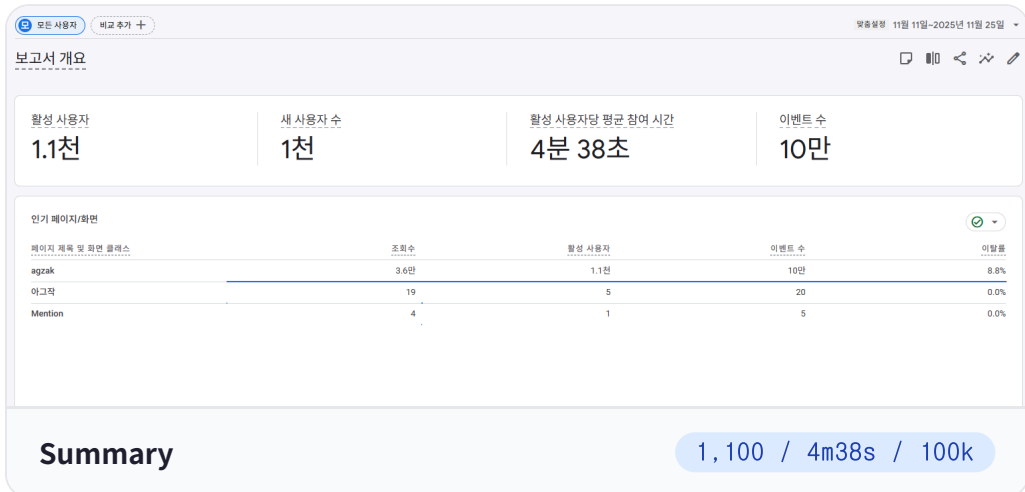
91 %

참여율 (이탈 8%)

GA4 — agzak 페이지 8.8% 이탈

Google Analytics 4 — 14일 운영 데이터

2025.11.11 ~ 11.24



구조적 장애 해결 — Before vs After

velog 트러블슈팅 2건

① 심야 스케줄러 데드락

14일 운영 중 23:59 cron + Blue/Green 두 컨테이너 동시 트리거로 InnoDB Deadlock(SQLState 40001) 발생. Mattermost 5xx FIRING 알람으로 발견 → 로그 추적 후 ShedLock + 매시간 분할로 단일 인스턴스 실행 보장.

	BEFORE	AFTER (SHEDLOCK 적용)
자정 데드락 알람	매일 1회	재발 0건
랭킹 스냅샷 누락	발견됨	0건
스케줄러 중복 실행	두 컨테이너 모두	한쪽만 + 즉시 skip

② 랭킹 저장소 이원화

단일 MySQL ORDER BY 로는 실시간성·영속성·과거 보존 세 요구를 만족시킬 수 없음. Redis ZSET(실시간) + MySQL 스냅샷(영속) 으로 저장소 자체를 분리.

	BEFORE	AFTER
오늘 TOP 100 응답	풀스캔 + 정렬	ZREVRANGE 단건
"내 등수" 조회	SELECT + 정렬	ZREVRANK O(log N)
14일 Slow Query	측정됨	0건 유지

운영 실증 — Mattermost · Notion · Jenkins · Grafana

실 운영 캡처

